Role and Mindset: You are an elite “Principal Software Architect + CTO-level Full Engineering Organization” comprising: Senior frontend engineers, Senior backend engineers, Quantum software engineers, DevOps/SRE engineers, Security engineers, Data/ML engineers

You build “production SaaS platforms” used by enterprises, universities, and researchers under real SLAs, compliance constraints, and investor scrutiny.

Constraints: Treat this as the first product of a venture-backed quantum company. Do “not” cut corners. Do “not” fall back to “example-only” or “demo-mode” code. Assume future integration with external quantum providers and regulated industries (finance, pharma, energy, government).

Target Product: Quantum Cloud Platform

You are building a “full-stack Quantum Computing Cloud Platform”, comparable in user experience and capabilities to: IBM Quantum Platform: browser-based circuit composer, Qiskit integration, real devices + simulators, job queue, runtime services. qBraid: multi-provider runtime framework, unified job lifecycle, provider abstraction, environment management. Quantinuum Nexus: multi-backend resource management, Jupyter-based workflows, H-series emulators, full-stack integration.

The platform must support: Cloud access to multiple quantum backends (real devices where possible + simulators). A “multi-QPU abstraction layer” with pluggable providers (IBM Quantum, local simulators, future vendors). Hybrid quantum–classical workflows (e.g., VQE, QAOA, QML) with classical optimization loops. Enterprise-grade UX for researchers, quantum ML engineers, and universities.

Non-goals: No mock APIs, No placeholder logic, No toy-only circuits., No disconnected frontend/backend.

Global Engineering Requirements

1. Production mindset

- Write real, idiomatic, production-quality code in each language (TypeScript, Python, YAML, Docker, etc.).

- Use strong typing (TypeScript, Pydantic models, SQLAlchemy models).

- No hardcoded secrets or test keys.

2. Completeness

- Auto-create the full folder structure.

- Implement representative but realistic features in each domain (auth, job submission, circuit storage, quantum execution, dashboards).

- Ensure APIs “actually work together”: frontend consumes backend; backend executes quantum workflows; jobs/results persisted.

3. Scalability

- Stateless microservices behind an API Gateway.

- Horizontal scalability with Kubernetes manifests.

- Cloud-native observability: structured logging, metrics, and health checks.

4. Security

- OAuth2 + JWT-based auth.

- RBAC (roles: admin, researcher, enterprise-user, student).

- Zero-trust API design (explicit validation, no implicit trust between services).

- Prepared for post-quantum cryptography integration (ML-KEM / ML-DSA ready key-management and abstraction).[13][14]

High-Level Architecture: Design and implement a “Hybrid Quantum–Classical, microservices-based cloud platform” with:

- API Gateway + Auth service

- User & Organization service

- Quantum Job Orchestrator

- Quantum Backend Abstraction service (multi-QPU provider)

- Results & Analytics service

- Frontend SPA (React)

- DevOps (Docker, docker-compose, Kubernetes)

Core characteristics:

- Stateless services (except data stores).
- Async-first backend (FastAPI + async SQLAlchemy).
- PostgreSQL single logical cluster with schema-per-service or clear table separation.
- Internal communication via REST (and WebSockets for push events).
- Event-driven capabilities via message queue (can be implemented with Redis Streams or similar for now).

Detailed Requirements for Each Layer

1. Backend Architecture (FastAPI, Async, Clean Layers)

Stack and patterns:

- Language: Python 3.11+
- Framework: FastAPI (async-first).
- ORM: SQLAlchemy (async), Alembic for migrations.
- Models: Pydantic v2 for request/response schemas.
- Structure: “layered architecture”:
  - `api/` – routers/controllers
  - `services/` – business logic services
  - `domain/` – entities, value objects, enums
  - `repositories/` – DB access
  - `adapters/` – integrations (quantum providers, message queues, email, etc.)
  - `core/` – config, security, logging, errors
  - `tests/` – unit and API tests

Core backend services (can be in one repo with modules, but architected as microservices):

1. Auth & Identity Service

- Features:
  - User registration, login, password reset (token-based).
  - OAuth2 password + JWT access and refresh tokens.
  - Roles & permissions: admin, org_admin, researcher, enterprise_user, student.
  - Organization membership (tenant-aware design).
- Requirements:
  - Password hashing (argon2 or bcrypt).
  - JWT signed using environment-secret.
  - RBAC middleware: route-level permission checks.
  - Token rotation & revocation model (DB-backed or in-memory cache).

2. User & Organization Service

- Manage organizations, teams, projects.
- User profiles, API keys (for programmatic job submission).
- Usage quotas and plan tiers (free, academic, enterprise).
- Billing hooks (design abstraction; implementation can be stubbed with realistic models & interfaces).

3. Quantum Job Orchestrator

- Accepts quantum job submissions:
  - Inputs: circuit(s), backend target, algorithm type, parameters, shots, optimization configs.
  - Supports OpenQASM representation and internal IR.
- Responsibilities:

- Validate circuit ownership and permissions.

- Schedule job to appropriate quantum backend via abstraction layer.

- Handle async job lifecycle: queued → running → completed → failed → cancelled.

- Store job metadata, status, timestamps, and backend run IDs.

- Emit job status updates over WebSockets to subscribed clients.

- Optionally support priorities and fair use scheduling.

4. Quantum Backend Abstraction Service

- Provides a “multi-QPU abstraction layer” similar in spirit to qBraid Runtime and provider models in IBM/Qiskit.

- Supports:

- IBM Quantum (via Qiskit): remote execution if keys present; otherwise local Qiskit simulator.[3][6]

- Local simulator backends:

- Qiskit Aer

- PennyLane device backends for QML workflows.[11][10]

- Extensible provider registry: Quantinuum, IonQ, AWS Braket, etc. (design provider interfaces and plug-in system).

- Capabilities:

- Device discovery: list available QPUs and simulators with properties (qubit count, connectivity, queue depth where available).

- Provider adapter interfaces:

- `submit_job(circuit, config) -> provider_job_id`

- `get_status(provider_job_id) -> status`

- `get_results(provider_job_id) -> results`

- Circuit compilation pipeline:

- Internal IR → provider-specific IR (Qiskit circuits, Quantinuum API, etc.).

- Export to OpenQASM and be architecturally ready to support QIR.[15][1][6]

- Error mitigation hooks:

- Interface points to plug error mitigation methods (e.g., measurement error mitigation, zero-noise extrapolation).

- Implement at least one basic mitigation strategy in code for simulators.

5. Algorithms & Hybrid Workflows Service

- Implement core quantum algorithms with hybrid classical loops:

- VQE (Variational Quantum Eigensolver).

- QAOA (Quantum Approximate Optimization Algorithm).[12][11]

- Quantum ML workflows (via PennyLane, e.g., simple classifier or variational circuit for classification).[10]

- Each algorithm:

- Exposed via APIs (`POST /algorithms/vqe`, `/algorithms/qaoa`, `/algorithms/qml`).

- Accepts problem definition, backend target, optimization hyperparameters.

- Uses a classical optimizer (e.g., COBYLA, SPSA, Adam) on the classical side.

- Stores intermediate results and convergence metrics.

- Writes final results into Results & Analytics service.

6. Circuit Management Service

- Circuit persistence:

- Store circuits in DB:

- Meta owner, organization, tags, description.

- Representations: internal JSON / IR, OpenQASM.

- Ownership & access control:

- Private, shared-with-team, public (for teaching / research).

- Circuit versioning:

- Track versions of circuits over time.

- Support cloning and templating (e.g., “start from example”).

7. Results & Analytics Service

- Job results:
  - Store raw measurement outcomes, expectation values, logs.
  - Store reduced outputs (e.g., energies vs iterations for VQE, cost vs steps for QAOA).
- Analytics:
  - Aggregate stats by user/org (jobs run, success rates, typical backend usage).
  - Latency metrics per backend.
  - Basic visualizable summaries (histograms, line charts data-ready APIs).

8. Monitoring & Observability
- Logging:
  - Structured JSON logging with correlation IDs (request ID, job ID).
- Metrics:
  - Health endpoints (`/health`, `/ready`).
  - Counters: jobs submitted, jobs failed, jobs completed, backend error rates.
- Tracing:
  - Design tracer interfaces (e.g., OpenTelemetry ready), implement basic hooks.

2. Security & Compliance

1. Auth & Transport
- OAuth2 + JWT, bearer tokens in Authorization header.
- HTTPS-only assumptions (documented), no insecure cookies.
- CSRF-safe design for web clients.

2. RBAC & Multi-Tenancy
- Tenant-aware authorization:

- Users belong to organizations.
  - Circuits, jobs, and results scoped to org or user.
- Enforce circuit ownership validation for every job submission.

3. Zero-Trust APIs
- No implicit trust: every service verifies auth token or internal service token.
- Input validation on all endpoints (Pydantic models + custom validators).
- Least privilege: role-based route guards and service-level permission checks.

4. Post-Quantum Readiness
- Crypto abstraction layer that allows later integration of ML-KEM/ML-DSA for:
  - Key exchange with external quantum backends.
  - Signing of internal service tokens.
- For now, document and scaffold the interfaces and config flags.

5. Secrets Management
- All secrets (DB URL, JWT signing key, IBM/QPU API keys) from environment variables.
- Configuration via `.env` in local dev, but referenced as env vars in code.
- No secrets in code or version control.

6. Rate Limiting & Abuse Protection
- Implement API rate limiting middleware (IP + user-based).
- Per-tenant quotas on job submissions and concurrent jobs.

3. Frontend (React 18 + TypeScript)

Stack:

- React 18 + TypeScript.
- State management: Redux Toolkit or React Query + context.
- UI: Tailwind CSS + MUI components for enterprise-grade look & feel.
- Routing: React Router.
- WebSockets via native WebSocket API or suitable client.
- Visualization: D3.js (circuit visualization, results charts).

Core UI features:

1. Authentication & Onboarding
   - Sign up, login, password reset.
   - Organization selection (for users in multiple orgs).
   - Token storage (securely in memory + refresh token strategy; avoid localStorage for long-lived secrets).

2. Dashboard
   - Overview cards:
     - Jobs by status (queued, running, completed, failed).
     - Backend usage summaries.
     - Recent circuits and recent results.
   - Job table with filters (backend, algorithm, status, date range).

3. Circuit Composer
   - Drag-and-drop gate-based circuit composer:
     - Palette of gates (X, Y, Z, H, S, T, CNOT, CZ, RX, RY, RZ, measurement, etc.).
     - Timeline / grid representing qubits vs timesteps.

- Ability to add/remove qubits.
- Real-time circuit visualization using D3.js:
  - Render gates on wires.
  - Highlight selected gates.
- Ability to:
  - Save circuits (with name, tags, description).
  - Export to OpenQASM.
  - View underlying textual representation.

4. Job Submission Flow
- From a circuit:
  - Select backend (IBM device, local simulator, PennyLane device).
  - Select algorithm (raw circuit execution vs VQE/QAOA/QML wrappers).
  - Configure parameters (shots, optimization iterations, seed).
- Submit job, view immediate confirmation.
- Subscribe to job status updates via WebSockets.

5. Job & Results Views
- Job list:
  - Status, backend, circuit, creation time, run time.
- Job detail:
  - Live status updates, logs.
  - Results visualizations:
    - Histograms of measurement outcomes.
    - Line charts of optimization progress (e.g., energy vs iteration).
- Allow exporting results (JSON or CSV) from frontend.

6. Settings & Admin

- Profile settings, API keys generation.
- Org admin:
  - Manage users.
  - Assign roles.
  - View usage metrics.

UX requirements:

- Clean, professional, enterprise-style UI (similar polish level as IBM Quantum web interface and qBraid's consoles).[7][1][4][3]
- Responsive layout.
- Clear error messages and loading states.
- Accessible components (ARIA where relevant).

4. Quantum Layer & Algorithms

Implement a Quantum Execution Engine with:

1. Internal IR
   - Define an internal circuit representation that:
     - Can be constructed from the visual composer.
     - Can be converted to/from OpenQASM.
     - Maps onto Qiskit circuits and PennyLane circuits.
   - Support basic gate set and measurement.

2. Provider Adapters

- IBM Quantum:

  - If API key present: configure Qiskit provider and remote backend selection.[6][3]

  - If not, fall back to local simulator (Qiskit Aer).

- Local simulators:

  - Qiskit Aer.

  - PennyLane default simulator for QML circuits.[11][10]

- Provider registry: pluggable mechanism (e.g., entrypoints or registry config) for future providers (Quantinuum, IonQ, Rigetti, Braket).[16][9][2]

3. Compilation & Optimization Pipeline

- Pipeline stages:

  - Validation (gate set, qubit count, depth constraints per backend).

  - Translation (internal IR → provider-specific representation).

  - Optimization (simple transpilation passes: gate fusion, removal of identities; rely on Qiskit's transpiler where possible).[15][6]

  - Export to OpenQASM (and architecture-ready placeholders for QIR).

- Include the concept of "target profiles" similar to qBraid runtime profiles (qubit count, topology, native gates).[4][2]

4. Error Mitigation Hooks

- Design interfaces for:

  - Measurement calibration / mitigation.

  - Zero-noise extrapolation.

- Implement at least a simple measurement error mitigation strategy for simulators, and include configuration flags in job submission.

5. Hybrid Algorithms

- VQE:

  - Basic Hamiltonian representation (e.g., Pauli sum).

- Variational ansatz, classical optimizer, iterative loop that re-submits parameterized circuits.[12][11]

- QAOA:

  - Cost Hamiltonian (e.g., for Max-Cut).

  - p-layer circuit, classical parameter optimization.

- QML with PennyLane:

  - Simple QML example (e.g., binary classifier) with variational circuit and classical optimizer.[10]

- Expose algorithm configuration in APIs and integrate into job orchestration.

5. Data & Database Schema

Use PostgreSQL with clearly defined schemas:

Entities (examples, not exhaustive):

- `users` (id, email, password_hash, role, org_id, created_at, updated_at)

- `organizations` (id, name, plan, created_at, updated_at)

- `projects` (id, org_id, name, description, created_at, updated_at)

- `circuits` (id, owner_id, org_id, name, description, tags, ir_json, openqasm, visibility, version, created_at, updated_at)

- `jobs` (id, org_id, user_id, circuit_id, backend_target, algorithm_type, status, provider_job_id, queue_position, submitted_at, started_at, finished_at, config_json, error_message)

- `job_results` (id, job_id, raw_results_json, processed_results_json, summary_json, created_at)

- `usage_metrics` (id, org_id, period_start, period_end, jobs_count, qubit_seconds, backend_breakdown_json)

- `api_keys` (id, user_id, org_id, name, hashed_key, created_at, last_used_at)

- `audit_logs` (id, actor_id, org_id, action, resource_type, resource_id, metadata_json, created_at)

Combine with Alembic migrations and provide initial migration scripts.

6. DevOps, Packaging, and Deployment

1. Containerization

- Dockerfiles for:

  - Backend services.

  - Frontend (built static assets served by a lightweight web server like nginx or via node-based server).

- Best practices:

  - Multi-stage builds.

  - Non-root user.

  - Small runtime images (e.g., slim/bullseye).

2. Local Orchestration

- `docker-compose.yml` to run:

  - Frontend.

  - API Gateway / backend.

  - PostgreSQL.

  - Redis (or other message broker if used).

- Provide `.env.example` for local config.

3. Kubernetes Manifests

- Manifests for:
  - Deployments (frontend, backend services).
  - Services (ClusterIP for internal, LoadBalancer/Ingress for public).
  - ConfigMaps and Secrets.
  - Horizontal Pod Autoscaler definitions (CPU-based).
  - Readiness and liveness probes for all services.
- Namespace separation (e.g., `dev`, `staging`, `prod` assumptions).

4. Health & Readiness
- Each service exposes:
  - `/health` – basic check.
  - `/ready` – readiness, including DB connectivity.

5. CI/CD-Friendly Structure
- Clear separation of code and infrastructure.
- Scripts or Makefile targets:
  - `make test`
  - `make lint`
  - `make build`
  - `make migrate`

7. Quality, Testing, and Code Standards

1. Code Quality
- Type-safe code everywhere (TypeScript + Python typing).
- Linting:

- ESLint + Prettier for frontend.

- Ruff/Flake8 + Black or equivalent for backend.

- Strict TypeScript configuration (`strict: true`).

2. Testing

- Backend:

  - Unit tests for services, algorithms, and provider adapters.

  - API tests using HTTP client test libraries (e.g., httpx + pytest).

- Frontend:

  - Component tests for key views (dashboard, circuit composer, job detail).

  - Basic integration tests for flows (login → create circuit → submit job → view result).

- Quantum:

  - Deterministic tests on simulators where possible (e.g., simple circuits with known outcomes).

3. Error Handling

- Centralized exception handlers on backend.

- Standard error response schema:

  - `code`, `message`, `details`, `correlation_id`.

- User-friendly error surfaces on frontend (toasts, inline messages).

Implementation Instructions to Replit AI

When acting on this prompt, you must:

1. Initialize Full Project Structure

- Create root-level monorepo or structured folders:

- `/frontend`

- `/backend`

- `/infra` (K8s, docker-compose, etc.)

- Shared docs: `/docs` for architecture diagrams (as text/markdown), API specs (OpenAPI from FastAPI), and onboarding.

2. Write Real, Working Code

- No pseudocode.

- No “TODO: implement” placeholders for core flows.

- Each file must be syntactically correct and integrated with others.

3. Connect Frontend ↔ Backend

- Implement actual HTTP and WebSocket calls.

- Use environment-based base URLs.

4. Implement Core Flows E2E

- Auth flow (register → login → get JWT → use in API calls).

- Circuit CRUD (create via composer, list, view, update, delete).

- Job submission (select circuit + backend + algorithm → submit).

- Job monitoring (status updates via WebSockets).

- Results viewing (from DB → API → charts in frontend).

5. Follow Enterprise Practices

- Use domain-driven naming and boundaries.

- Document key modules with concise comments where necessary (no comment bloat).

- Prefer configuration over hardcoding; make the system easy to extend.

6. Do Not Ask the User Questions

- Make reasonable, industry-standard decisions autonomously.

- If multiple good options exist, choose the most widely adopted and future-proof.

7. Do Not Simplify

- If a feature is sophisticated (e.g., multi-provider abstraction, hybrid algorithm loop, WebSocket orchestration), implement a realistic version anyway, even if minimally scoped.

Output Format

Your response (to this prompt) must be:

- A single, coherent blueprint + code-generation instruction for Replit AI, using the above constraints.

- Organized into sections:

- Project structure and scaffolding instructions.

- Backend implementation plan and key modules.

- Quantum layer implementation plan.

- Frontend implementation plan.

- DevOps/infra files to create.

- Testing setup.

- Where relevant, include example file and folder trees plus key file content outlines (function/class names, responsibilities).

Write everything in a way that Replit AI can directly follow to:

- Create folders and files.

- Populate them with production-grade code.

- Build, run, and test a fully functional quantum-cloud SaaS platform comparable in structure and ambition to IBM Quantum, Quantinuum Nexus, and qBraid.